Traffic Sign Geometric Shape and Colour Analysis

Detecting and classifying road signs with colour and geometry rules only

HSV masking | Douglas-Peucker polygons | No artificial intelligence | Python + OpenCV + NumPy

PROJECT TEAM

Berat Yaylacı
Student No: 230212056

Yusuf Emre Eroğlu
Student No: 240212006

1. What does the project do?

Road signs follow strict **rules**: each class has a characteristic **colour** (red, blue, yellow) and a characteristic **shape** (octagon, triangle, circle, square, diamond). The program reads a street photo, finds regions of those colours, traces their outlines, and counts the **corners** of each outline to decide what the shape is.
Colour + shape together give the sign class, for example **red + octagon = STOP**, **blue + circle = mandatory**. Objects that have the right colour but the wrong shape, such as a red car, are **rejected**. On our demo scene all **7 signs** were classified correctly and **2 look-alikes** were rejected.

2. Aim, motivation and where it is used

Aim: to detect and classify traffic signs using only classical image processing, as the course requires: **no AI, no training data**.

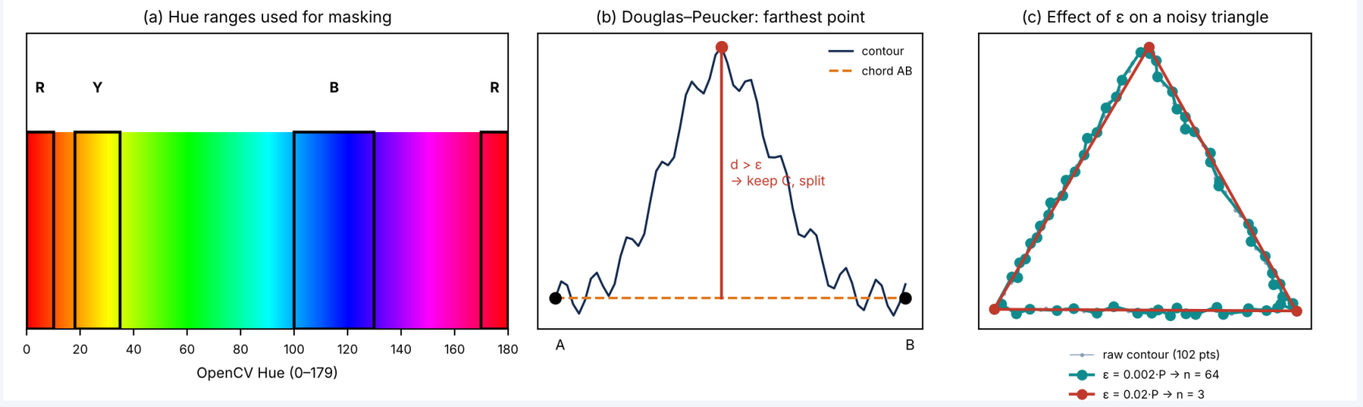
- **Autonomous-driving assistance** — speed-limit and stop-sign warnings for the driver.
- **Smart-city infrastructure** — automatic inventory of signs from street-level cameras.
- **Road-safety audits** — find faded, missing or wrongly placed signs.
- **Fast & explainable** — a few OpenCV calls; every decision is a visible rule.

3. The mathematics: HSV colour space and Douglas-Peucker polygon approximation

Why HSV, not RGB? In RGB, a red sign in shadow changes all three channels. HSV separates **Hue** (the colour itself) from **Saturation** and **Value** (brightness), so one hue range works in sun and shade:

V = max(R,G,B), S = (V − min(R,G,B)) / V

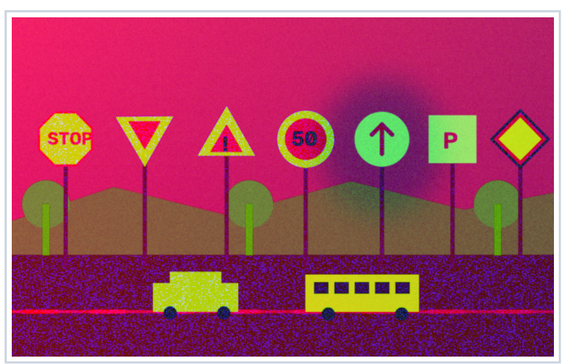
Douglas-Peucker (`cv2.approxPolyDP`): take the chord AB of a contour piece, find the point farthest from it; if that distance d > ε keep the point and split, else drop all points between. We use **ε = 0.02 · perimeter**. The number of corners n that remains gives the shape: **n = 3** triangle, **n = 4** square / diamond, **n = 8** octagon.



4. How it works — the pipeline, shown on a real run of our code (synthetic street scene with uneven lighting and a shadow)



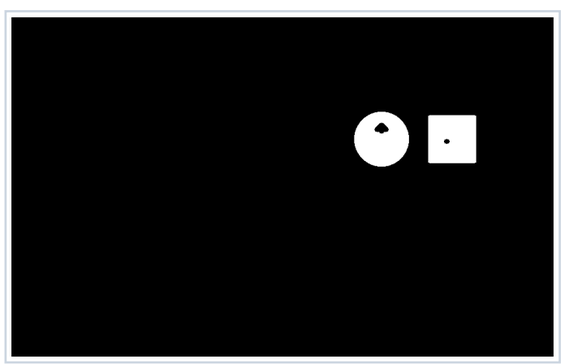
STEP 1 · Input: 7 signs, a red car and a yellow bus; shadow on the 4th/5th sign



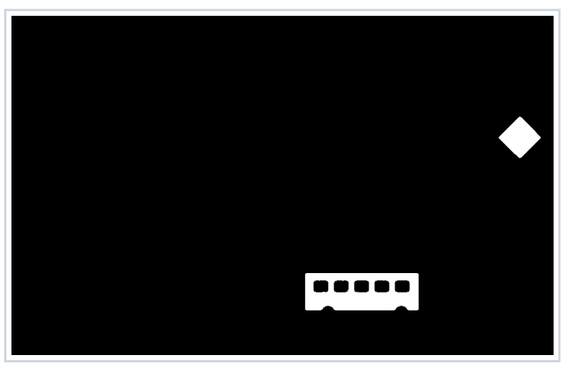
STEP 2 · BGR → HSV (H,S,V shown as colour): signs have high saturation



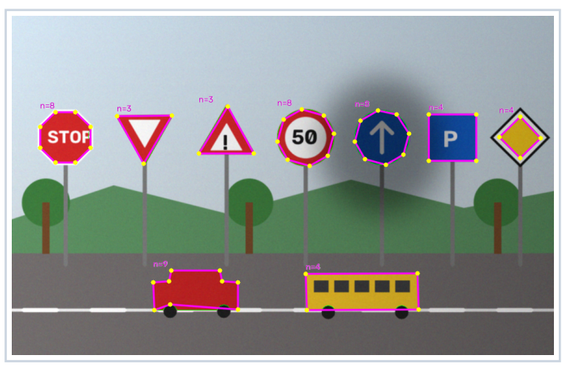
STEP 3 · Red mask (H 0–10 and 170–179): STOP, YIELD, WARNING, 50-limit, car



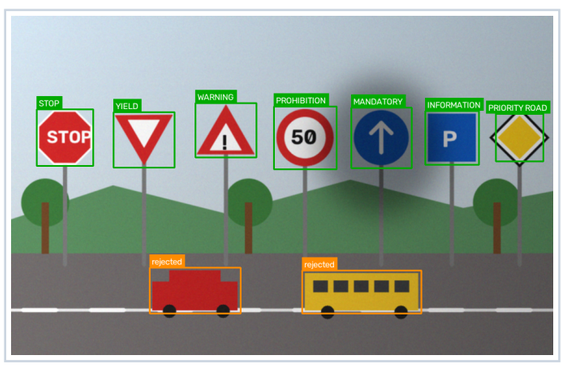
STEP 4 · Blue mask (H 100–130): mandatory and information signs



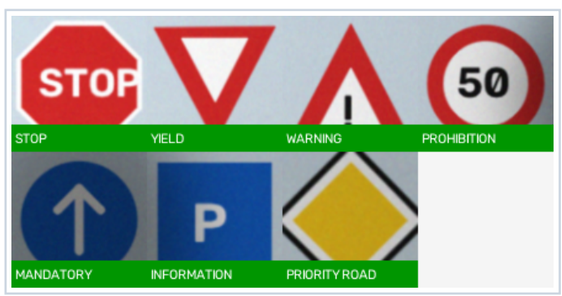
STEP 5 · Yellow mask (H 18–35): priority road sign and the bus



STEP 6 · findContours + approxPolyDP (pink polygon, yellow = corners, n = count)



STEP 7 · Rules applied: green = classified sign, orange = rejected



STEP 8 · The 7 detected signs cropped out with their predicted class

5. Decision rules and measured values (from this run)

Fill ratio = contour area / area of its minimum enclosing circle. Circle ≈ 1.0, octagon ≈ 0.90, square ≈ 0.64, triangle ≈ 0.41.

Detected object	Colour mask	Corners n	Fill ratio	Rule that fires	Result
STOP	red	8	0.89	n ≈ 8, aspect ≈ 1, fill 0.80–0.94 → octagon	✔ STOP
Yield	red	3	0.44	n = 3, apex pointing down	✔ YIELD
Warning	red	3	0.45	n = 3, apex pointing up	✔ WARNING
Speed limit 50	red	8	0.98	fill > 0.94, aspect ≈ 1 → circle	✔ PROHIBITION
Arrow sign	blue	8	0.98	fill > 0.94, aspect ≈ 1 → circle	✔ MANDATORY
P sign	blue	4	0.65	n = 4, aspect ≈ 1, edges axis-aligned	✔ INFORMATION
Diamond sign	yellow	4	0.67	n = 4, aspect ≈ 1, edges at 45°	✔ PRIORITY ROAD
Red car	red	9	0.46	aspect 2.13: not a sign shape	✘ rejected
Yellow bus	yellow	4	0.37	aspect 3.05: not a square/diamond	✘ rejected

6. Tools & implementation

```
hsv = cv2.cvtColor(img, BGR2HSV)
m = cv2.inRange(hsv, lo, hi) (R, B, Y)
m = open + close (ellipse kernel)
cnts = cv2.findContours(m, EXTERNAL)
ap = cv2.approxPolyDP(c, 0.02*P, True)
n = len(ap); fill = A / (π r²)
label = rule(colour, n, fill, aspect)
```

Python · OpenCV · NumPy · HSV masking · approxPolyDP

7. Results on the demo scene

7 / 7 signs classified correctly (covering 5 shape types: octagon, triangle, circle, square, diamond).
2 / 2 look-alikes rejected: red car and yellow bus have the right colour but the wrong geometry.
The shadowed signs (4th, 5th) were still found, because HSV keeps hue stable when brightness drops.

8. Limitations & future work

- Circle vs. octagon: corner count alone is ambiguous (both gave n = 8), so the **fill ratio** is used as a tie-breaker.
- Faded, occluded or very small signs, and red objects near a sign, can break masks.
- Next: read the symbol inside (digits, arrows) by template matching, and test on real video.

9. References

1. Ramer, U. (1972). An iterative procedure for the polygonal approximation of plane curves. Computer Graphics and Image Processing, 1(3), 244-256.
2. Douglas, D. H., & Peucker, T. K. (1973). Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. The CanCart, 10(2), 112-122.
4. De La Escalera, A., Armingol, J. M., & Mata, M. (2003). Traffic sign recognition and analysis for intelligent vehicles. Image and Vision Computing, 21(3), 247-258.
5. OpenCV Documentation. (2026). Contours Hierarchy and Polygon Approximation (approxPolyDP) Guide. docs.opencv.org